

文章笔记

文章笔记：Harness Engineering for Coding Agents

基于 HumanLayer 技术博客整理

2026 年 4 月 2 日

“The model is probably fine. It’s just a skill issue.”

— HumanLayer

文章作者：HumanLayer

发布日期：2025

预计阅读时长：约 25 分钟

文章链接：<https://www.humanlayer.dev/blog/skill-issue-harness-engineering-for-coding-agents>

项目主页：<https://github.com/hqh1025/ai-course-notes>

目录

1 引言：不是模型的问题，是配置的问题	3
1.1 本章小结	3
2 Harness Engineering 与 Context Engineering	3
2.1 Harness Engineering 的定义	3
2.2 Context Engineering 的超集关系	4
2.3 Post-Training 与 Harness 的耦合	4
2.4 本章小结	5
3 CLAUDE.md 与 AGENTS.md: Agent 配置文件	5
3.1 基本概念	5
3.2 ETH Zurich 研究的启示	5
3.3 编写高质量 Agentfile 的原则	5
3.4 本章小结	6
4 MCP Servers: 工具扩展	6
4.1 MCP 的角色	6
4.2 工具过多的危害	6
4.3 CLI 优于 MCP 的场景	7
4.4 实战案例：Linear CLI 替代 MCP	7
4.5 本章小结	7
5 Skills: 可复用知识的渐进式披露	7
5.1 Skills 的概念	7
5.2 渐进式披露的核心思想	8
5.3 Skill 的激活机制	8
5.4 Skills 与工具分发	8
5.5 本章小结	8
6 Sub-Agents: 上下文隔离的利器	8
6.1 Sub-Agents 的正确用法	8
6.2 Context Rot: 经验证据	9
6.3 为什么“更大的 Context Window”不是答案	9
6.4 Sub-Agent 的典型用例	10
6.5 Sub-Agents 与成本控制	10
6.6 自制 Sub-Agent 方案	10
6.7 本章小结	10
7 Hooks: 确定性控制流	11
7.1 Hooks 的概念	11
7.2 Hooks 的典型用例	11
7.3 实战 Hook 示例	11
7.4 本章小结	12

8 Back-Pressure: 自动验证机制	12
8.1 核心洞察	12
8.2 验证机制的类型	12
8.3 验证机制的 Context Efficiency	13
8.4 本章小结	13
9 实践经验: 什么有效, 什么无效	13
9.1 不要过度优化	13
9.2 无效的做法	13
9.3 有效的做法	14
9.4 本章小结	14
10 总结与延伸	14
10.1 核心要点回顾	14
10.2 贯穿全文的设计理念	14
10.3 拓展阅读	15

1 引言：不是模型的问题，是配置的问题

HumanLayer 团队在过去一年中目睹了 coding agent 以各种方式失败：忽视指令、未经提示执行危险命令、在最简单的任务上反复打转。每次失败后，本能的反应总是一样的：“我们只需要更好的模型”、“GPT-6 会修好一切”、“等训练数据里有了我用的那个库就好了”。

然而，经过数十个项目、数百次 agent 会话的实践，HumanLayer 反复得出同一个结论：**这不是模型问题 (model problem)，而是配置问题 (configuration problem)。**

核心论点

Coding agent 的质量公式：

$$\text{Coding Agent} = \text{AI Model(s)} + \text{Harness}$$

模型只是其中一半。Harness（运行时环境与配置）决定了模型能力能否被充分发挥。与其祈祷下一代模型拯救一切，不如专注于“如何从当前模型中榨取最大价值”。

是的，模型会越来越聪明，某些现有的失败模式会消失。但正因为模型更聪明了，我们会给它们更大、更难的新问题，而它们将以意想不到的方式继续失败。**意外的失败模式是非确定性系统的根本问题。**

什么是 Harness？

Harness 是 coding agent 的“运行时”或“外围设备”：它是模型与环境交互的所有配置点的总和。包括但不限于：skills、MCP servers、sub-agents、memory、AGENTS.md 文件等。你可以把它理解为模型的“装备系统”—模型本身是角色属性，harness 是武器、护甲和工具。

1.1 本章小结

Coding agent 的表现不仅取决于底层模型的能力，更取决于 harness 的质量。HumanLayer 的核心主张是：当 agent 失败时，先检查 harness，再怀疑模型。

2 Harness Engineering 与 Context Engineering

2.1 Harness Engineering 的定义

Harness engineering 这个术语由 Viv 提出，其核心思想是：**每当你发现 agent 犯了一个错误，就花时间工程化一个解决方案，使得 agent 永远不会再犯同样的错误。**（Mitchell Hashimoto 的总结）

Harness Engineering 的六个核心问题

1. 如何赋予 coding agent 新的能力?
2. 如何教会它训练数据中没有的代码库知识?
3. 如何在 system message 中的 “CRITICAL: always do XYZ” 之外增加确定性?
4. 如何让 agent 的行为适配特定代码库?
5. 如何在 “魔法 prompt” 之外提高任务成功率?
6. 如何防止 context window 过快膨胀或被低质量上下文填满?

2.2 Context Engineering 的超集关系

HumanLayer 的联合创始人 Dex 在 12-factor agents 中提出了 context engineering 的概念。Context engineering 是 prompt engineering 的超集，涵盖了系统性提高 AI agent 可靠性的各种技术。

Harness engineering 则是 context engineering 的子集，主要通过 harness 配置点来精心管理 coding agent 的 context window。

Viv 的四个定制杠杆

Viv 关于 harness engineering 的文章将定制杠杆分为四类：

1. **System prompt**：系统级指令
2. **Tools / MCPs**：工具与能力扩展
3. **Context**：上下文管理
4. **Sub-agents**：子代理分工

HumanLayer 额外强调了两个 Viv 未重点讨论的杠杆：**hooks**（自动化集成与确定性控制流）和 **skills**（渐进式知识披露）。

2.3 Post-Training 与 Harness 的耦合

前沿 coding 模型会在其对应 harness 上进行 post-training（例如 Claude 在 Claude Code 上训练，GPT-5 Codex 在 Codex harness 上训练）。这导致了有趣的现象：

Post-Training 的双刃剑

模型对其训练 harness 的过度拟合可能适得其反。Terminal Bench 2.0 的数据显示：Opus 4.6 在 Claude Code 中排名第 33，但放到一个 post-training 时未见过的 harness 中，排名升至第 5（上下浮动约 4 名）。这说明 **默认 harness 不一定是最优 harness**，定制化配置可能释放模型被默认 harness 压制的潜力。

一个典型案例是 OpenCode 项目。Codex 模型与 Codex harness 的 `apply_patch` 工具紧密耦合，以至于 OpenCode（一个开源的 Claude Code 替代方案）不得不专门为 GPT/Codex 模型添加

`apply_patch` 工具来模拟 Codex harness, 而 Claude 和其他模型则继续使用常规的 `edit` 和 `write` 工具。

2.4 本章小结

Harness engineering 是 context engineering 的子集, 通过配置 harness 的各个组件来管理 agent 的 context window。核心理念是: 遇到错误就工程化一个永久性修复。即使模型经过了 post-training, 默认 harness 也并非不可改进。

3 CLAUDE.md 与 AGENTS.md: Agent 配置文件

3.1 基本概念

在触碰任何其他 harness 配置点之前, 通常值得先定制你的 CLAUDE.md / AGENTS.md 文件。这些是放在仓库根目录的 markdown 文件, harness 会确定性地将其注入 agent 的 system prompt。

3.2 ETH Zurich 研究的启示

ETH Zurich 发表了一项研究, 测试了 138 个 agentfile 在多个仓库中的效果, 结论是大多数 agentfile “无用甚至有害”。研究的主要发现:

表 1: ETH Zurich Agentfile 研究关键发现

发现	具体数据
LLM 生成的文件	实际损害表现, 且成本增加 20%+
人工编写的文件	仅帮助约 4%
推理 token 消耗	Agent 多花 14-22% 的推理 token 处理指令
步骤与工具调用	更多步骤、更多工具调用, 但分辨率未提升
代码库概览	目录列表完全无帮助, agent 自己就能发现仓库结构

3.3 编写高质量 Agentfile 的原则

HumanLayer 认为该研究恰好验证了他们之前的建议:

Agentfile 最佳实践

- 1. 避免自动生成: LLM 生成的 agentfile 效果更差
- 2. 指令越少越好: 不省略必要指令的前提下, 尽量精简
- 3. 使用渐进式披露 (Progressive Disclosure): 不要把所有信息都塞进去
- 4. 保持简洁和普适: 内容应普遍适用, 避免过多条件规则
- 5. 不要过度引导工具使用: 过度指定 “用什么工具” 反而导致更差的结果

HumanLayer 自己的 CLAUDE.md 不到 60 行。

常见误区

- 把代码库概览和目录结构放进 agentfile — agent 自己会探索
- 用 LLM 自动生成 agentfile — 实际效果为负
- 塞入大量条件规则 (“如果 X 则 Y, 否则 Z”) — 增加推理负担却不提高成功率
- 详细指定每个任务应该使用什么工具 — 过度约束模型的灵活性

3.4 本章小结

CLAUDE.md / AGENTS.md 是最基础的 harness 配置点。但“有配置”不等于“好配置”：自动生成的、信息过载的、过度引导的文件反而有害。核心原则是精简、普适、渐进式披露。

4 MCP Servers: 工具扩展

4.1 MCP 的角色

MCP (Model Context Protocol) servers 主要用于将工具 (tools) 接入 coding agent, 扩展其文件 I/O 和 bash 命令之外的能力。MCP 规范还包括 resources、prompts、elicitations 等特性, 但这些在 MCP 客户端和 coding agent harness 中通常支持不佳。

当你将 MCP server 接入 coding agent 时, 可用工具列表、工具描述和调用参数会被注入 agent 的 system prompt。因此, MCP server 可以通过工具描述来定制 agent 的行为。

MCP 安全警告

因为 MCP server 的工具描述会被添加到 coding agent 的 system prompt 中, **永远不要连接你不信任的 MCP server**。这可能成为 prompt injection 的危险载体。STDIO servers 和其他通过 `npx` 或 `uvx` 在客户端运行的 server 也可以在没有 prompt injection 的情况下在你的主机上执行代码。

4.2 工具过多的危害

HumanLayer 的亲身经验: 接入过多 MCP 工具后, context window 被工具描述填满, agent 更快地进入“笨区” (dumb zone)。

Instruction Budget 概念

每个 agent 的 context window 有一个隐式的“instruction budget” (指令预算)。每一条无关的工具描述都是 agent 必须处理但没有收益的指令。这些无用指令会挤占真正重要的上下文空间。Anthropic 甚至为此发布了实验性的 MCP tool search 功能, 在用户连接了过多 MCP 工具时渐进式地向 Claude 披露工具。

关键建议: 如果你不在主动使用某个提供大量工具的 MCP server, 就关掉它。

4.3 CLI 优于 MCP 的场景

HumanLayer 发现，当 MCP server 复制了训练数据中已有 CLI 的功能时，直接提示 agent 使用 CLI 效果更好。对于 GitHub、Docker 或大多数数据库，coding agent 可以直接使用对应的 CLI 和 shell 命令。模型在训练时已经见过这些工具，且 CLI 可以与 `grep`、`jq` 等工具组合使用，提供额外的上下文效率。

4.4 实战案例：Linear CLI 替代 MCP

HumanLayer 曾使用 Linear MCP server，后来发现实际只用到其中一小部分功能。于是他们编写了一个轻量 CLI 来封装 Linear API，提供非常精简的响应，并在 CLAUDE.md 中给出 6 条用法示例。

```
1 ## Linear
2 Use the Linear CLI for:
3 - fetching issues:    linear get-issue ENG-XXXX
4 - listing issues:    linear list-issues or linear my-issues
5 - adding comments:   linear add-comment -i ENG-XXXX "comment"
6 - adding links:      linear add-link ENG-XXXX "url" -t "title"
7 - updating status:   linear update-status ENG-XXXX "status name"
8 - get branch name:   linear get-issue-v2 ENG-XXXX --fields branch
9 - get ticket images: linear fetch-images ENG-XXXX
```

Listing 1: CLAUDE.md 中的 Linear CLI 示例配置

这种方式节省了数千个 token（原本来自 MCP server 的工具定义），以及更多来自冗长 MCP 响应的 token。

4.5 本章小结

MCP servers 是扩展 agent 能力的强大工具，但需要克制使用。工具过多会挤占 context window、增加无效指令负担。当 CLI 已经在训练数据中被充分覆盖时，CLI 是更好的选择。始终以 context efficiency 为优化目标。

5 Skills：可复用知识的渐进式披露

5.1 Skills 的概念

Skills 最初由 Anthropic 为 Claude Code 引入，现已成为多个 harness（如 Codex、OpenCode）支持的开放标准。每个 skill 是一个目录，包含一个 SKILL.md 文件和可能的附属资源。

Skills 安全风险

Skill 注册中心已经被发现分发过数百个恶意 skill。像对待 `npm install random-package` 一样对待 skill — 阅读你正在安装的内容。ClawHub、skills.sh 等注册中心的 skill 可以在你的机器上执行任意代码。

5.2 渐进式披露的核心思想

HumanLayer 早期犯过的错误：把所有指令和工具都塞进 system prompt，结果 agent 越来越差。在 agent 开始工作之前，instruction budget 就已经被消耗殆尽。

Skills 通过渐进式披露解决这个问题：agent 只在决定需要时（或你替它决定时）才获得特定的指令、知识或工具。

5.3 Skill 的激活机制

当 skill 被激活时，其目录中的 SKILL.md 文件作为 user message 加载到 agent 的 context window 中。SKILL.md 文件可以告知 agent 该 skill 还捆绑了什么其他资源。

表 2: Skill 目录结构示例

文件/目录	用途
SKILL.md	主指令文件，描述 skill 的功能和使用方式
response_template.md	响应模板，规范 agent 的输出格式
CLIs/linear-cli	捆绑的命令行工具
CLIs/tunnel-cli	捆绑的命令行工具

更高级的渐进式披露：你可以在 skill 中捆绑多个 markdown 文件，每个文件包含不同功能或用途的信息，主 SKILL.md 文件告诉 agent 其他文件是什么以及何时应该读取它们。

5.4 Skills 与工具分发

目前无法直接在 skill 中捆绑 MCP server 或自定义 agent 工具。替代方案是将工具编写为可执行文件、CLI、NPM 包或其他可以与 skill 一起分发或在 skill 文件中指示 agent 安装的形式。

例如，与其配置一个 Playwright MCP server，不如提供一个基于 BrowserBase 的 agent browser skill 或 Vercel 的 agent browser CLI 的 web 浏览 skill。

5.5 本章小结

Skills 是实现渐进式披露的关键机制—只在需要时加载知识和工具，避免 system prompt 过载。使用时需注意安全风险，避免盲目安装未审核的第三方 skill。

6 Sub-Agents：上下文隔离的利器

6.1 Sub-Agents 的正确用法

Sub-agents 是一个流行但常被误解的 harness 配置点。HumanLayer 尝试过“前端工程师”sub-agent、“后端工程师”sub-agent、“数据分析师”sub-agent 的模式。**这不管用。**

Sub-Agents 的核心价值：Context 隔离

Sub-agents 真正有效的用法是 **context control**（上下文控制）。它们提供了一种封装整个 coding agent 会话工作的方式：调度 agent 只看到它写给 sub-agent 的 prompt 和 sub-agent 的最终结果。所有中间的工具调用、工具返回结果或其他消息都不会进入父 agent 的 context window。

将工作拆解为离散任务并委派给 sub-agents，是保持主 agent 线程处于“聪明区”的关键方法。

6.2 Context Rot：经验证据

Chroma 的 context rot 研究为 HumanLayer 的实践经验提供了实证支持：**模型在更长的 context length 下表现更差。**

关键研究发现：

- 测试了 18 个模型的 needle-in-a-haystack 任务
- 随着 context 长度增加，性能下降——即使在简单任务上
- 当问题与上下文中相关信息的语义相似度低时，性能下降更陡峭
- 干扰效应在更长的 context window 中会复合叠加

在实践中，HumanLayer 观察到：父会话中每一个不相关的中间工具调用、每一个 grep 结果、每一次文件读取，都是潜在的干扰源。

6.3 为什么“更大的 Context Window”不是答案

对“扩大 Context Window”方案的质疑

当一个实验室提供某模型的 extended-context 版本时，你通常得到的不是一个具有更大“instruction budget”的更大模型，而是**同一个模型加上一些巧妙的数学技巧**（如 YaRN）来延长模型能处理的序列长度。

回到 needle-in-a-haystack 问题：更大的 context window 并不能让模型更擅长找针——**它只是让干草堆更大了**。如果你认为需要更长的 context，你可能只是需要**更好的 context window 隔离**。

Sub-agents 从结构上解决这个问题：每个 sub-agent 获得一个全新的、小型的、高相关性的 context window 和全新的 instruction budget，只有压缩后的结果流回父 agent。这让你可以将多个 context window 串联起来解决一个大问题。

6.4 Sub-Agent 的典型用例

表 3: Sub-Agent 适用场景

用例	说明
定位代码定义/实现	问题简单但需要大量中间工具调用来搜索
分析代码库模式	识别特定类型工作的代码模式
跟踪信息流	例如跨服务边界追踪一个请求的流转
代码/文档/Web 研究	通用研究任务，结果简洁但过程繁重

Sub-agents 应返回高度压缩的响应，同样遵循渐进式披露原则。例如，HumanLayer 的 sub-agents 在回答问题的同时引用源（文件路径: 行号或 URL），这样父 agent 不会暴露于 sub-agent 使用的所有源，但如果需要更多细节，它有足够的信息去找到相关上下文。

6.5 Sub-Agents 与成本控制

Sub-agents 还能帮助控制成本。HumanLayer 对父会话使用昂贵模型 (Opus)，因为计划和编排等任务需要深度思考；对每个 sub-agent 使用更便宜、更快的模型（如 Sonnet 或 Haiku）。Sub-agents 接收的是更小、更离散的任务，低阶模型即可胜任——不需要在一次代码搜索上消耗 Opus 的 token。

6.6 自制 Sub-Agent 方案

当 Harness 不原生支持 Sub-Agents 时

有些 harness 完全不支持 sub-agents（甚至 Codex 直到最近才添加，且支持仍是实验性的）。替代方案：编写一个 MCP server，提供一个工具来启动新的 agent 会话。该工具接收父 agent 的 prompt，启动一个新的 coding agent 会话，然后返回 sub-agent 的最终响应给父 agent。需要注意的陷阱：

- 如果 harness 原生支持 sub-agents，这种模式会允许 sub-agents 通过 MCP 再分派 sub-agents，可能导致不可预测的“电话游戏”
- 必须非常仔细地编写 sub-agent 的 system prompt，明确指定其角色范围：做什么、不做什么、返回什么信息、如何返回、有哪些工具
- 许多 harness 有 MCP 工具调用超时限制，可能需要调高

6.7 本章小结

Sub-agents 的核心价值不是角色分工，而是 context 隔离。它们通过为每个离散任务创建独立的 context window，防止中间噪声在父线程中累积。同时，sub-agents 还提供了成本控制的手段，对不同复杂度的任务使用不同级别的模型。

7 Hooks: 确定性控制流

7.1 Hooks 的概念

Claude Code 有 hooks 的概念：用户定义的命令或脚本，在特定事件发生时和 agent 生命周期的各个阶段自动执行。类似地，OpenCode 有 plugins。其他 coding agent 可能有类似的配置点（遗憾的是，Codex 没有等效功能）。

Hooks 在概念上类似于 git hooks，但更灵活。它们可以用来添加新功能、集成外部服务、自动化常规操作、修改权限和配置默认行为。

7.2 Hooks 的典型用例

表 4: Hooks 常见用例

用例类别	具体实现
通知	Agent 完成时播放声音；审批待处理过久时提醒
审批	基于输入值和更灵活的规则自动批准或拒绝工具调用。例如：自动拒绝所有试图运行数据库迁移的 <code>Bash()</code> 调用，并指示 agent 让用户手动执行
集成	Agent 完成时发送 Slack 消息、创建 GitHub PR、设置预览环境
验证	每次 agent 停止时运行类型检查或构建，将错误反馈给 agent

7.3 实战 Hook 示例

以下是 HumanLayer 仓库中使用的一个 Stop hook。当 Claude 停止时，它运行 Biome 格式化工具和 TypeScript 类型检查。有错误则反馈给 Claude；无错误则静默退出。

```
1 #!/bin/bash
2 cd "$CLAUDE_PROJECT_DIR"
3
4 # prebuild: generate types and build internal SDK
5 PREBUILD_OUTPUT=$(bun run generate-cache-key \
6   && turbo run build --filter=@humanlayer/hld-sdk \
7   && bun install 2>&1)
8 if [ $? -ne 0 ]; then
9   echo "prebuild failed:" >&2
10  echo "$PREBUILD_OUTPUT" >&2
11  exit 2
12 fi
13
14 # biome and typecheck run in parallel
15 # biome --write exits code 1 if it made changes,
16 # so run twice: second pass exits 0 if all fixed
17 OUTPUT=$(bun run --parallel \
18   "biome check . --write --unsafe \
19   ||| biome check . --write --unsafe" \
```

```

20 "turbo_run_typecheck" 2>&1)
21 if [ $? -ne 0 ]; then
22     echo "$OUTPUT" >&2
23     exit 2
24 fi

```

Listing 2: HumanLayer 的验证 Hook (Stop Hook)

Hook 设计原则：成功静默，失败详报

这个 hook 体现了一个关键设计原则：

- **成功时完全静默**—没有任何内容进入 agent 的 context
- **失败时只输出错误**—仅将必要信息反馈给 agent
- **exit 2 告诉 harness 重新启动 agent**，让它在完成之前修复错误

这与 back-pressure 的理念一脉相承：让 agent 能够验证自己的工作，而不是把验证留给人类。

7.4 本章小结

Hooks 提供了在 agent 生命周期的特定节点注入确定性控制流的能力。最有价值的模式是验证 hook：成功静默、失败时才向 agent 反馈错误信息，迫使 agent 在标记完成之前解决问题。

8 Back-Pressure：自动验证机制

8.1 核心洞察

HumanLayer 的核心洞察：**使用 coding agent 成功解决问题的概率，与 agent 验证自身工作的能力强相关。**

他们在构建测试和其他 back-pressure 机制上投入了大量时间，这仍然是投资回报率最高的工作之一。

8.2 验证机制的类型

HumanLayer 代码库中的验证机制包括：

表 5: Back-Pressure 验证机制

机制	说明
类型检查和构建步骤	最好使用强类型语言
单元测试/集成测试	验证功能正确性
代码覆盖率报告	有一个 Stop hook 在覆盖率下降时提示 agent 增加覆盖率
UI 交互和测试集成	Playwright、agent-browser 等

8.3 验证机制的 Context Efficiency

验证输出必须精简——血泪教训

HumanLayer 早期让 agent 在每次改动后运行完整的测试套件，结果 4000 行通过的测试输出淹没了 context window。Agent 随后失去了对实际任务的跟踪，开始对它刚刚读取的测试文件产生幻觉。

现在的做法：**吞掉成功输出，只呈现错误**。构建也是一样——**成功是静默的，只有失败才产生详细输出**。

验证机制必须是 context-efficient 的，否则验证本身就成了干扰源。

HumanLayer 在 CLAUDE.md 文件中给 Claude 提供了关于如何使用所有这些验证机制的简洁指令，有些甚至被打包进 skills 中以实现渐进式披露。

8.4 本章小结

Back-pressure 是提高 agent 任务成功率的最高杠杆点之一。关键是让 agent 能自我验证，但验证输出必须精简：成功静默，失败详报。否则验证本身会污染 context window，反而降低 agent 的表现。

9 实践经验：什么有效，什么无效

9.1 不要过度优化

HumanLayer 坦诚承认：完全有可能花在优化 coding agent 配置上的时间比实际用 agent 写代码的时间还多——他们经历过。他们的做法是：**偏向交付** (bias towards shipping)。只在 harness 配置确实能帮助更快交付更高质量代码的程度上投入时间。

当 agent 失败时，花时间工程化一个解决方案使其不再以同样方式失败——但不会预防性地去寻找问题来解决。

9.2 无效的做法

表 6: HumanLayer 实践中无效的做法

做法	问题
提前设计理想 harness	在遇到真实失败之前做的配置往往是错误的
安装大量 skills 和 MCP servers “以防万一”	增加无效指令负担，挤占 context
每次会话后运行完整测试套件 (5+ 分钟)	输出淹没 context window
微观优化 sub-agents 的工具访问权限	导致大量工具来回切换，效果更差

9.3 有效的做法

经过验证的最佳实践

1. **从简单开始**: 只在 agent 实际失败时才添加配置
2. **设计-测试-迭代-丢弃**: HumanLayer 丢弃的 hooks 远多于实际使用的
3. **仓库级配置分发**: 将经过实战检验的配置通过仓库级别的配置文件分发给整个团队
4. **优化迭代速度**: 不要追求“第一次就对”，而是让每次迭代更快
5. **先扩展再收缩**: 先给 agent 一整套能力（如 Linear），然后在了解实际需求后精简暴露给模型的内容

9.4 本章小结

Harness engineering 的最佳策略是反应式而非预防式：从简单开始，在失败中学习，逐步添加配置。同时要有勇气丢弃不管用的东西。优化迭代速度比追求一步到位更重要。

10 总结与延伸

10.1 核心要点回顾

这篇 HumanLayer 的文章系统性地论述了 coding agent 性能优化的方法论。核心论点可以浓缩为一句话：“**The model is probably fine. It’s just a skill issue.**”——当你的 coding agent 表现不佳时，问题大概率出在 harness（配置与运行环境），而非模型本身。

文章介绍的六大 harness 配置杠杆可以总结如下：

表 7: 六大 Harness 配置杠杆总览

配置杠杆	核心作用	关键原则
CLAUDE.md AGENTS.md	/ 基础行为配置	精简、普适、避免自动生成
MCP Servers	能力扩展	按需启用、CLI 优先
Skills	渐进式知识披露	按需加载、注意安全
Sub-Agents	Context 隔离	用于隔离而非角色分工
Hooks	确定性控制流	成功静默、失败详报
Back-Pressure	自动验证	验证输出必须精简

10.2 贯穿全文的设计理念

文章反复强调的几个设计理念值得深思：

1. **Context Efficiency**: 所有配置决策的终极评判标准是“这是否让 context window 中的信息更高效？”
2. **Progressive Disclosure**: 信息按需加载，避免预加载所有可能需要的内容

3. **Silent Success, Verbose Failure**: 成功不产生输出，失败才详细报告—这是验证机制的设计金律
4. **Reactive, Not Proactive**: 不要预防性优化，在真实失败中学习和改进
5. **Instruction Budget**: 将 context window 视为有限预算，每条指令都有机会成本

10.3 拓展阅读

- HumanLayer 关于 CLAUDE.md 最佳实践的博文
- Dex 的 12-factor agents 框架
- Viv 关于 harness engineering 的系列文章
- ETH Zurich 的 agentfile 有效性研究（138 个 agentfile 的实证分析）
- Chroma 的 context rot 研究
- Terminal Bench 2.0 benchmark
- Matt Pocock 关于 AGENTS.md 的文章
- OpenAI 关于 harness engineering 的博文